PROJECT

CSE0002

OPEN SOURCE SOFTWARE (LINUX ADMINISTRATION)

SLOT-B22

SUBMITTED BY:

Name : Anushka Chaudhary

Reg no. : 24BCY10208

SUBMITTED TO:

Dr. Pradeep Kumar Mishra

(Assistant Professor Grade 2)

School Of Computing Science Engineering and

Artificial Intelligence (SCAI)

VIT Bhopal University

Date of Submission:

31/03/2026

TABLE OF CONTENT

1. **Introduction**

2. **Part-A - Origin And Philosophy**

A1. The Problem This Software Was Created to Solve

A2. The License — What It Actually Says

A3. The Ethics of Open Source

3. **Part B — Linux Footprint**

4. **Part C — The FOSS Ecosystem**

5. **PART D - OPEN SOURCE VS PROPRIETARY**

6. **Project Timeline**

7. **REFERENCES**

8. **CONCLUSION**

9. **SHELL SCRIPTS**

9.1 Script 1

9.2 Script 2

9.3 Script 3

9.4 Script 4

9.5 Script 5

INTRODUCTION

In this digital era, software plays a significant role in our daily life in terms of education, communication, business, and technology. Open Source Software (OSS) is an essential element in modern computer systems because it provides users with access to freely usable source code that they can modify and share with other users.

Unlike other software developed and owned by companies, open-source software provides users with more flexibility and transparency. Users can modify the software according to their requirements without relying on a single company. This provides users with more reliable and cost-effective software.

Linux and Python are popular technologies used in open-source software. Linux is an operating system used in servers, cloud computing, and mobile devices. Python is a popular programming language known for its simplicity and flexibility in programming. This combination provides a strong foundation for Free and Open Source Software (FOSS).

This project is designed to explore the concept of open-source software in terms of its origin, philosophy, and impact on society, as well as its comparison with other software systems in terms of its impact on the future of technology.

PART-A - ORIGIN AND PHILOSOPHY

A1. The Problem This Software Was Created to Solve

Before the development of Git, it was a very difficult and inefficient task to manage the source code for software development. Most programmers used centralized version control tools like CVS and Subversion. In centralized version control, all the files related to a particular project were stored centrally. This led to a number of issues. In case of a server crash, the whole project could be compromised. In addition, programmers had to be online for most of the tasks. This made the whole process very slow and internet-dependent.

Another problem faced by programmers during this time, especially for large projects, was the issue of collaboration. In a large team of programmers, it was very difficult for programmers to work on the same files. This led to a number of complications. As projects grew larger, so did the issues faced by programmers. The tools they were using were not designed for this high-speed development, where programmers from all over the world were contributing.

These problems posed a significant challenge, and this was very apparent in the development of the Linux kernel, which had thousands of contributors worldwide. There was a need for a system that could handle large-scale development efficiently, enable multiple people to work independently, and also provide a complete and accurate record of the development process.

Git was created to solve all these problems by providing a distributed version control system. This means that each developer has a complete copy of the entire history of the project. This allows developers to work independently without relying on a central server. This also makes it much faster since most of the work happens locally.

Git also provides many features for branching and merging, allowing developers to work on multiple features simultaneously without affecting the main code. This also makes it much easier to merge changes later on. Furthermore, Git provides data integrity through secure hashing techniques, making it a very reliable system.

Git has therefore solved all the major problems associated with speed, collaboration, reliability, and scalability in software development. It has therefore become a fundamental tool for modern programming.

A2. The License — What It Actually Say

Licensing is one of the most critical aspects of open-source software, as it determines the way in which the software is utilized, altered, and distributed. Without a licensing mechanism, there would be ambiguity in terms of ownership and usage rights.

One of the most prominent open-source software licenses is the GNU General Public License. This license is critical in ensuring that the open-source software is free and open for all to utilize. With the help of the GPL, users can modify and distribute the open-source software, while the modified version is also shared in the same way. This is called "copyleft." This concept is critical in ensuring the freedom of open-source software.

On the other hand, there are other open-source software licenses, such as the MIT License and Apache License. These licenses are critical in providing flexibility to users, as they can modify the open-source software and utilize it for proprietary purposes.

A3. The Ethics of Open Source

Open source is not just technology; it is also values and ethics. Open source is founded on the belief that knowledge must be freely shared among people. This encourages a culture of learning and mutual growth among people.

The ethical values of open source include freedom, transparency, collaboration, and accessibility. Freedom allows users to have full control of the applications they use. Transparency allows users to have access to the source code of the applications they use, thus encouraging security. Collaboration allows programmers all over the globe to collaborate and make applications better. Accessibility allows all people to have access to technology, irrespective of their financial status.

Organizations such as the Free Software Foundation and the Open Source Initiative have supported and advocated for the development of open source technology all over the globe. These ethical values make open source technology a powerful force in society.

PART B — LINUX FOOTPRINT

It has become one of the most widely used and influential operating systems in the world. What started as a small open-source project has now become a powerful platform for a large part of the world's technology.

One of the largest spaces in which Linux has a significant presence is in the world of web servers and the internet. A large percentage of websites around the world run on Linux-based servers. This is because Linux is stable, secure, and can handle high traffic very efficiently. This makes Linux the backbone of the internet and everything related to it.

Another major space in which Linux is heavily used is in the world of cloud computing. Some of the largest names in cloud computing, such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud, run Linux-based systems. This is because Linux is open-source, and organizations can customize their systems according to their needs.

Another significant domain is mobile technology. The Android mobile phone operating system, used by most mobile phones around the world, is based on Linux. This is a testament to the flexibility and power of Linux, which can be used for everything from high-powered computer servers to small handheld devices.

Moreover, Linux is widely used in embedded systems and IoT devices. Smart TVs, routers, and other devices run Linux. Linux is very lightweight and flexible, making it perfect for devices like these.

Distributions like Ubuntu, Debian, and Fedora, among others, vary in terms of the features they support. Ubuntu is best for new users, Debian is best for stability, and Fedora is best for power users and developers.

Another major feature of Linux is its support for programming and automation. It supports languages like Python, which are used for scripting in Linux. This makes tasks very efficient and eliminates the need for manual intervention.

To sum it all, Linux is a massive technology domain, and its applications are in many areas, such as servers, cloud computing, mobile devices, and embedded systems. Linux is open-source, stable, and flexible, and its presence is felt in all aspects of modern computing.

The following screenshots confirm Python installation and configuration on the Linux system:

```
anushka@DESKTOP-0JMC594:~$ python3 --version
Python 3.12.3
```

```
anushka@DESKTOP-0JMC594:~$ which python3
/usr/bin/python3
anushka@DESKTOP-0JMC594:~$ command -v python3
/usr/bin/python3
```

```
anushka@DESKTOP-0JMC594:~$ python3 -c "print('Python is working')"
Python is working
anushka@DESKTOP-0JMC594:~$
```

PART C — THE FOSS ECOSYSTEM

The FOSS ecosystem is a complex and ever-expanding network of individuals, organizations, tools, and communities. It is a decentralized and community-driven process. In this process, anyone with the appropriate skills and interest is allowed to contribute. It is more open, collaborative, and innovative compared to the traditional process.

One of the most crucial parts of the FOSS ecosystem is the presence of version control systems, especially Git. It helps the developers track changes to their code. It allows the developers to have different versions of a project. It facilitates the efficient collaboration of individuals. Platforms such as GitHub and GitLab are central nodes for the FOSS ecosystem. Developers can host projects, contribute to open-source projects, and collaborate with other developers. It facilitates collaboration more effectively than ever.

Another crucial part of the FOSS ecosystem is the presence of open-source programming languages and tools. Some of the most used programming languages are Python, Java, and C++. Among these, Python plays a crucial role. It is easy to learn and use. It is used for various applications. Some of the most used applications are web development, data science, artificial intelligence, machine learning, and automation. Python is used for a wide variety of applications. It is coupled with various libraries and tools. Some of the most used tools are NumPy, Pandas, TensorFlow, etc.

The FOSS ecosystem is also greatly supported by a global community of developers. This is because they assist in developing code, correcting bugs, improving documentation, testing software, and disseminating knowledge through tutorials and online forums. This ensures that open-source software is constantly improved over time.

Another significant feature of the FOSS ecosystem is its accessibility and cost-effectiveness. This is because most open-source software is freely available to users, making it easier for students, new businesses, and small organizations to access it without incurring significant costs.

The FOSS ecosystem also plays a significant role in education and skill development. This is because students and other developers can access free resources, documentation, and projects to gain more knowledge and experience in

software development. The open-source platforms offer an opportunity to learn through practice rather than learning in theory.

In addition, the FOSS ecosystem is significant in that it promotes innovation and quick development in the ecosystem. This is because there is a collaborative effort among different developers in the ecosystem. This leads to the generation of new ideas and quick solutioning of problems in the ecosystem. This is evident in the many modern technologies and innovations developed using the ecosystem.

In conclusion, it is evident that the FOSS ecosystem is a powerful ecosystem that encourages learning, innovation, and global participation in software development.

PART D - OPEN SOURCE VS PROPRIETARY

DIMENSION	PYTHON (OPEN SOURCE)	PROPRIETARY SOFTWARE
Cost	Completely free to use, no licensing fees	Requires paid licenses or subscriptions
Security	Source code is open and can be audited by anyone	Closed source, users must trust the company
Support	Community support, forums, documentation	Official paid support from company
Freedom To Modify	Full freedom to modify and customize	No permission to modify source code
Community vs Corporate Control	Community-driven development	Controlled by a single company
Flexibility	Highly flexible and adaptable	Limited flexibility

The fact that Python is a free and open-source programming language also means that users do not have to pay any fees whatsoever in order to utilize this programming language. This is in direct contrast to other types of software that require users to pay a significant amount of money in order to access their services.

One of the most important reasons why Python stands out as a superior programming language is its transparency. The fact that Python's source code is freely available means that users can learn how it works and even make changes to it in order to make it even better. This also gives Python an edge over other types of software in terms of security because any bugs found in it can be fixed immediately by people around the world. This contrasts directly with other types of software whose source code is not freely available and whose security entirely depends on the company that developed it.

In terms of support, Python also stands out as a superior programming language because it has an enormous community that provides support to its users through online platforms and other collaborative platforms. This contrasts directly with other types of software whose official support is not reliable and requires users to pay an extra fee in order to access it..

PROJECT TIMELINE

The project was carried out in a well-planned and organized way over a period of time. In the beginning, the fundamental concepts of Linux and FOSS were studied to develop a clear understanding. After that, focus was shifted to learning Python and basic shell scripting skills. In the following stage, different scripts were created and tested to check their accuracy and performance. In the final stage, the complete report was prepared by combining both theoretical knowledge and practical work in a proper and systematic format..

REFERENCES

- https://www.kernel.org
- https://www.gnu.org
- https://opensource.org
- https://www.python.org
- https://docs.github.com
- https://www.geeksforgeeks.org

CONCLUSION

In conclusion, this project helped gain a better understanding of Linux and Free and Open Source Software (FOSS) and their significance in modern computing. The significance of open-source software, which promotes transparency, collaboration, and innovation through free access and utilization of source code, can be understood through this project. The study of Linux and Python helped gain a better understanding of their significance as tools for modern computing, especially in the fields of system management, software development, and automation. The significance of open-source licenses and ethical values, which ensure fairness, freedom, and accessibility for all, can also be understood through this project. The contrast between open-source and proprietary software helped gain a better understanding of the significance of open-source software, which is more flexible and cost-effective compared to proprietary software. The implementation of shell scripting helped gain hands-on experience, which is very useful for improving problem-solving skills. This project not only helped gain a better theoretical understanding of the topic but also helped gain a better practical understanding of the topic. This shows that open-source technologies will play a very significant role in shaping the future of computing.

SHELL SCRIPTS

Script 1 — System Identity Report

Purpose

The purpose of this script is to generate a detailed system identity report that displays important information about the user and the operating system. It collects data such as the system kernel version, current user, system uptime, date and time, and operating system details. This script helps in understanding the system environment and provides a quick overview of system configuration, which is useful for auditing and documentation purposes.

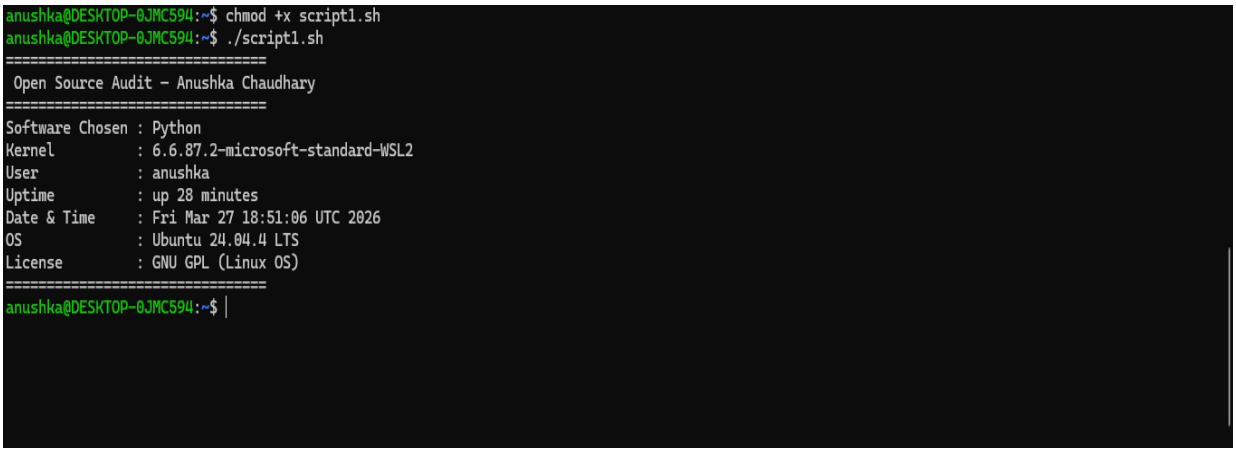
Concepts Used

This script uses basic shell scripting concepts such as **variables, command substitution, and system commands**. Commands like `uname`, `whoami`, `uptime`, and `date` are used to retrieve system information, while `/etc/os-release` is accessed to get OS details. The script also demonstrates the use of `echo` for formatted output. Overall, it shows how shell scripting can be used to collect and display system-level information efficiently.

```
#!/bin/bash

# Script 1: System Identity Report

# Author: Your Name


STUDENT_NAME="Anushka Chaudhary"

SOFTWARE_CHOICE="Python"


KERNEL=$(uname -r)

USER_NAME=$(whoami)

UPTIME=$(uptime -p)

DATE=$(date)

DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2)
```

```
echo "======================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "======================================"
echo "Software Chosen : $SOFTWARE_CHOICE"
echo "Kernel          : $KERNEL"
echo "User            : $USER_NAME"
echo "Uptime          : $UPTIME"
echo "Date & Time     : $DATE"
echo "OS              : $DISTRO"
echo "License         : GNU GPL (Linux OS)"

echo "======================================"
```

screenshot:



Script 2 — Python Package Inspector

Purpose

The purpose of this script is to check whether a specific software package, particularly Python, is installed on the system. It verifies the availability of the package and displays its version if installed. Additionally, it provides a brief description of the package, helping users understand its purpose. This script is useful for ensuring that required tools are available before starting development work.

Concepts Used

This script uses **conditional statements and command checking**. The `command -v` command is used to check whether the package exists in the system. The `if-else` statement is used to handle different outcomes based on whether the package is installed or not. It also uses a `case` statement to display descriptions for different software packages. This demonstrates decision-making and branching in shell scripting.

```
#!/bin/bash

# Script 2: Python Package Inspector


PACKAGE="python3"


if command -v $PACKAGE C>/dev/null; then

  echo "$PACKAGE is installed."

  $PACKAGE --version

else

  echo "$PACKAGE is NOT installed."

fi
```

```
case $PACKAGE in
   python3) echo "Python: A powerful open-source programming language" ;;
   git) echo "Git: Version control system for developers" ;;
   firefox) echo "Firefox: Open-source web browser" ;;
   vlc) echo "VLC: Media player that plays everything" ;;
   *) echo "Unknown package" ;;
Esac
```

screenshot:

Script 3 — Disk and Permission Auditor

Purpose

The purpose of this script is to analyze system directories by checking their permissions and disk usage. It provides a detailed report of important directories such as `/etc`, `/var/log`, `/home`, and others. Additionally, it verifies the presence of the Python binary and displays its file details. This script is useful for system monitoring, security auditing, and understanding resource usage.

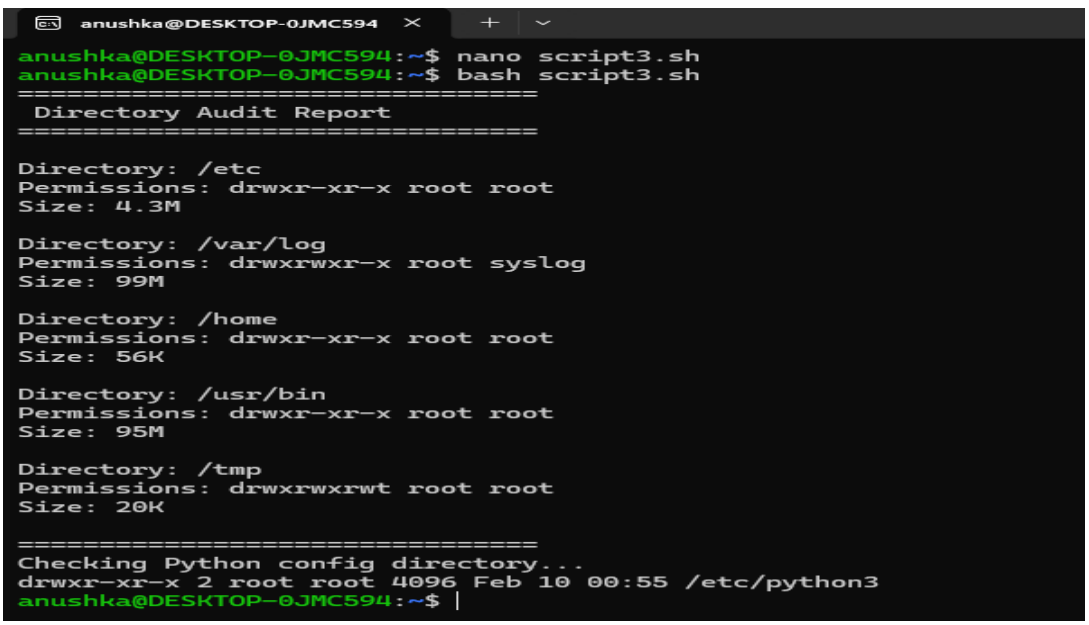
Concepts Used

This script demonstrates **loops, arrays, and file handling concepts**. It uses an array to store directory paths and a `for` loop to iterate through them. Commands like `ls -ld` are used to check permissions, and `du -sh` is used to calculate directory size. The script also uses conditional statements (`if`) to verify directory existence and file presence. This shows how shell scripts can automate repetitive system checks efficiently.

```
#!/bin/bash

# Script 3: Disk and Permission Auditor


DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")


echo "Directory Audit Report"

echo "--------------------------"


for DIR in "${DIRS[@]}"; do

  if [ -d "$DIR" ]; then

    PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')

    SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

    echo "$DIR => Permissions: $PERMS | Size: $SIZE"

  else

    echo "$DIR does not exist"

  fi

done
```

```
# Python directory check
PYTHON_PATH=$(which python3)


if [ -f "$PYTHON_PATH" ]; then
   echo "Python binary found at: $PYTHON_PATH"
   ls -l $PYTHON_PATH
else
   echo "Python not found"
fi
```

screenshot:



Script 4 — Log File Analyzer

Purpose

The purpose of this script is to analyze log files and count the occurrences of a specific keyword, such as “error”. It reads the log file line by line and identifies matching entries. It also displays the last few matching lines, which helps in debugging and monitoring system issues. This script is particularly useful for system administrators to quickly identify errors in large log files**Concepts Used**

This script uses **file handling, loops, and string processing techniques**. It accepts input arguments for the log file and keyword, making it flexible. A `while` loop is used to read the file line by line, and `grep` is used for pattern matching. Arithmetic operations are used to count occurrences. It also demonstrates the use of default values and input validation. This script highlights how shell scripting can be used for text processing and analysis.

```
#!/bin/bash

# Script 4: Log File Analyzer


LOGFILE=$1

KEYWORD=${2:-"error"}

COUNT=0


if [ ! -f "$LOGFILE" ]; then

  echo "Error: File not found."

  exit 1

fi


while IFS= read -r LINE; do

  if echo "$LINE" | grep -iq "$KEYWORD"; then

    COUNT=$((COUNT + 1))

  fi

done < "$LOGFILE"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
```

echo "Last 5 matching lines:"

grep -i "$KEYWORD" "$LOGFILE" | tail -5

screenshot:

```
anushka@DESKTOP-0JMC594:~$ nano script4.sh
anushka@DESKTOP-0JMC594:~$ bash script4.sh
==============================
 Log File Analyzer
==============================
Enter log file path (e.g. /var/log/syslog): /var/log/syslog
Enter keyword (default = error): error

Keyword 'error' found 11 times

Last 5 matching lines:
2026-03-27T18:35:25.703387+00:00 DESKTOP-0JMC594 wsl-pro-service[195]: #033[33mWARNING#033[0m Exiting after could not ensure valid Landscape configu
ration: could not register distro to Landscape: could not enable Landscape: /usr/bin/landscape-config: error: exit status 1.
2026-03-28T10:27:04.901783+00:00 DESKTOP-0JMC594 systemd[1]: apport-autoreport.path - Process error reports when automatic reporting is enabled (fil
e watch) was skipped because of an unmet condition check (ConditionPathExists=/var/lib/apport/autoreport).
2026-03-28T10:27:04.901788+00:00 DESKTOP-0JMC594 systemd[1]: apport-autoreport.timer - Process error reports when automatic reporting is enabled (ti
mer based) was skipped because of an unmet condition check (ConditionPathExists=/var/lib/apport/autoreport).
2026-03-28T10:27:04.902236+00:00 DESKTOP-0JMC594 kernel: RAS: Correctable Errors collector initialized.
2026-03-28T10:27:37.559600+00:00 DESKTOP-0JMC594 kernel: WSL (215) ERROR: CheckConnection: getaddrinfo() failed: -5
anushka@DESKTOP-0JMC594:~$
```

Script 5 — Open Source Manifesto Generator

Purpose

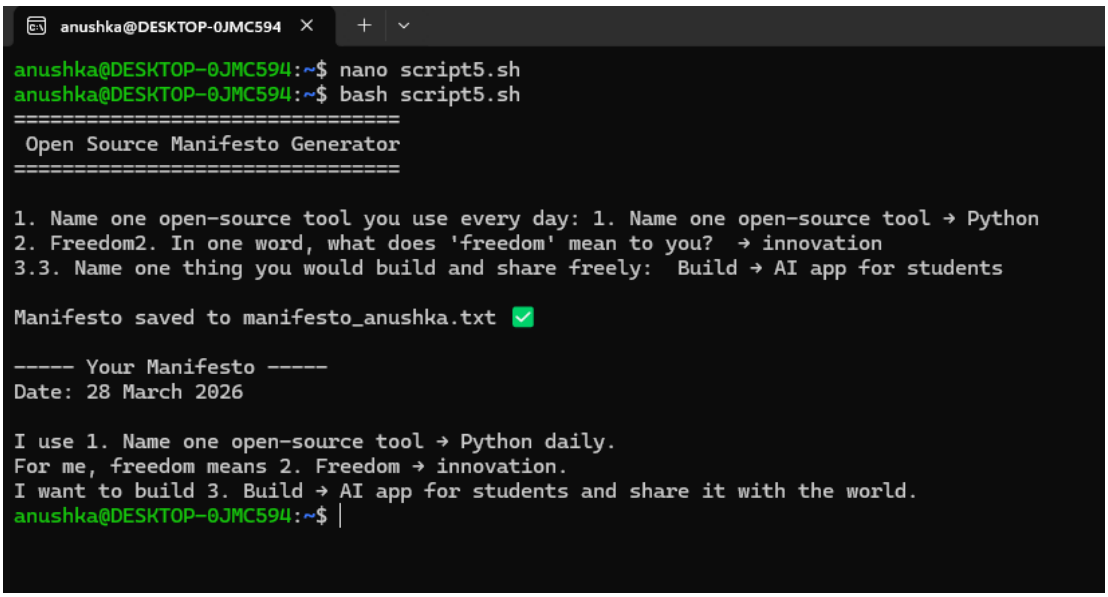
The purpose of this script is to generate a personalized open-source manifesto based on user input. It collects responses from the user about their daily tools, understanding of freedom, and future goals, and then creates a text file containing a structured message. This script promotes awareness of open-source philosophy and allows users to express their ideas creatively.

Concepts Used

This script uses **user input, file handling, and string formatting**. The `read` command is used to take input from the user, and variables are used to store responses. Output redirection (`>`, `>>`) is used to create and write content into a file. The script also uses the `date` command to include the current date in the output. This demonstrates how shell scripting can be used to interact with users and generate dynamic content.

```
#!/bin/bash

# Script 5: Open Source Manifesto Generator
echo "Answer the questions:"

read -p "1. Tool you use daily: " TOOL

read -p "2. Meaning of freedom: " FREEDOM

read -p "3. What will you build: " BUILD



DATE=$(date '+%d %B %Y')

OUTPUT="manifesto_$(whoami).txt"



echo "On $DATE, I believe in open source." > $OUTPUT

echo "I use $TOOL daily." >> $OUTPUT

echo "For me, freedom means $FREEDOM." >> $OUTPUT

echo "I want to build $BUILD and share it with the world." >> $OUTPUT
```

echo "Manifesto saved to $OUTPUT"

cat $OUTPUT

screenshot: